MOBL-12: Advanced Instrumentation & Optimization

> **Series:** MOBL | **Notebook:** 12 of 12 | **Created:** February 2026 | **Last Updated:** 02/24/2026

Overview

This notebook covers advanced mobile SDK techniques that go beyond automatic instrumentation. You will learn how to send custom business events, create manual and nested user actions, report errors and events programmatically, tag web requests for end-to-end tracing, instrument A/B tests and feature flags, tune SDK performance, and manage multi-app monitoring strategies. These techniques give you fine-grained control over what mobile telemetry reaches Grail and how it maps to your business logic.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
Permissions	`rum.read`, `bizevents.read`, `entities.read`
Mobile App	iOS or Android app with Dynatrace SDK integrated
Prior Knowledge	Familiarity with MOBL-01 through MOBL-09 (fundamentals, SDK setup, user actions, crash reporting, network monitoring)
SDK Version	Dynatrace iOS Agent 8.x+ or Android Agent 8.x+

1. Custom Business Events

The `sendBizEvent()` API allows you to send custom business events directly from your mobile app into Grail. Unlike auto-captured user actions, business events let you track domain-specific interactions that matter to your business -- purchases, sign-ups, feature usage, subscription changes, and more.

Why Custom Business Events?

Use Case	Example	Business Value
Purchases	In-app purchase completed	Revenue tracking, conversion funnels
Sign-ups	New account registration	Growth metrics, onboarding analysis
Feature usage	User opened a specific feature	Feature adoption, prioritization
Content engagement	Article read, video watched	Content strategy optimization
Error context	User-reported issue with metadata	Correlate user feedback with technical data

iOS Implementation

```
// iOS -- send a purchase business event
let attributes: [String: Any] = [
    "event.type": "com.myapp.purchase",
    "product.name": "Premium Subscription",
    "product.price": 9.99,
    "currency": "USD",
    "payment.method": "apple_pay"
]
Dynatrace.sendBizEvent(type: "com.myapp.purchase", attributes: attributes)
```

Android Implementation

```
// Android -- send a purchase business event
val attributes = mapOf(
    "event.type" to "com.myapp.purchase",
    "product.name" to "Premium Subscription",
    "product.price" to 9.99,
    "currency" to "USD",
    "payment.method" to "google_pay"
)
Dynatrace.sendBizEvent("com.myapp.purchase", attributes)
```

Best Practices for Business Events

- **Use reverse-domain naming** for `event.type` (e.g., `com.myapp.purchase`) to avoid collisions
- **Keep attribute names consistent** across iOS and Android implementations
- **Include version context** -- add `app.version` as an attribute if not automatically enriched
- **Avoid PII** -- never include email addresses, phone numbers, or other personal data in business event attributes

Query Custom Purchase Events

Use the following DQL query to retrieve and inspect custom purchase business events sent from your mobile app:

```
// Query custom purchase business events
fetch bizevents, from:-24h
| filter event.type == "com.myapp.purchase"
| fields timestamp, product.name, product.price, currency, payment.method
| sort timestamp desc
| limit 50
```

2. Advanced User Actions

While the Dynatrace SDK auto-instruments many user actions (taps, screen loads), advanced scenarios require manual control. This section covers nested actions, long-running actions, and action properties.

Nested Actions

Nested actions let you create parent-child relationships between user actions. This is useful for multi-step flows where you want to measure both the overall flow and individual steps.

```
// iOS -- nested actions for a checkout flow
let parentAction = DTXAction.enter(withName: "Checkout Flow")
let childAction = DTXAction.enter(withName: "Validate Cart", parentAction:
parentAction)
// ... validation logic ...
childAction.leave()
// ... more checkout logic ...
parentAction.leave()
```

```
// Android -- nested actions for a checkout flow
val parentAction = Dynatrace.enterAction("Checkout Flow")
val childAction = Dynatrace.enterAction("Validate Cart", parentAction)
// ... validation logic ...
childAction.leaveAction()
// ... more checkout logic ...
parentAction.leaveAction()
```

Long-Running Actions

By default, the SDK auto-closes actions after a timeout (typically 500ms of inactivity). For long-running operations like file uploads, background syncs, or multi-screen

wizards, you need to manage the action lifecycle explicitly:

```
// iOS -- long-running action for file upload
let uploadAction = DTXAction.enter(withName: "Upload Photo")
uploadPhoto { result in
    switch result {
    case .success:
        uploadAction.reportValue(withName: "upload.size_bytes", intValue:
fileSize)
        uploadAction.leave()
    case .failure(let error):
        uploadAction.reportError(withName: "upload.failed", error: error)
        uploadAction.leave()
    }
}
```

Action Properties

Attach custom properties to user actions for richer analysis:

Method	Type	Example
`reportValue(withName:intValue:)`	Integer	`action.reportValue(withName: "items_in_cart", intValue: 3)`
`reportValue(withName:doubleValue:)`	Double	`action.reportValue(withName: "total_price", doubleValue: 49.99)`
`reportValue(withName:stringValue:)`	String	`action.reportValue(withName: "category", stringValue: "electronics")`
`reportError(withName:error:)`	Error	`action.reportError(withName: "validation_error", error: err)`

> **Tip:** Action properties become queryable fields in Grail. Use consistent naming across platforms so your DQL queries work for both iOS and Android data.

3. Error & Event Reporting APIs

Beyond automatic crash detection, the SDK provides APIs to report handled errors, custom error conditions, and diagnostic events. This is critical for capturing errors that your app handles gracefully (try/catch) but that you still want visibility into.

Reporting Handled Errors

```
// iOS -- report a handled error
do {
    let data = try parseUserProfile(json)
} catch {
    Dynatrace.reportError(withName: "profile_parse_error", error: error)
}
```

```
// Android -- report a handled error
try {
    val data = parseUserProfile(json)
} catch (e: Exception) {
    Dynatrace.reportError("profile_parse_error", e)
}
```

Reporting Custom Events

```
// iOS -- report a custom event with context
Dynatrace.reportEvent(withName: "low_storage_warning", attributes: [
    "available_mb": availableMB,
    "threshold_mb": 100
])
```

Query Reported Errors

Use the following query to retrieve errors that were explicitly reported from your mobile SDK:

```
// Reported errors from mobile apps
fetch bizevents, from:-24h
| filter event.provider == "www.dynatrace.com/mobile"
| filter event.type == "com.dynatrace.error.report"
| fields timestamp, useraction.application, event.name, os.type, app.version
| sort timestamp desc
| limit 50
```

Error Reporting Best Practices

- **Report all handled exceptions** that indicate degraded functionality -- even if the user does not see an error message
- **Include context** -- attach relevant metadata (user action, screen name, request URL) to help with root cause analysis
- **Avoid high-volume reporting** -- do not report transient conditions (e.g., network retries) that self-resolve. Focus on errors that impact the user experience
- **Use consistent error names** across platforms so a single DQL query can aggregate iOS and Android errors together

4. Web Request Tagging

Web request tagging links mobile-initiated HTTP requests to their corresponding server-side traces. This enables true end-to-end distributed tracing from the user's device through your backend services.

How It Works

1. The mobile SDK generates a unique request tag for each outgoing HTTP request
2. Your app adds this tag as a custom HTTP header (`x-dynatrace`)
3. The Dynatrace server-side agent (OneAgent) reads the header and correlates the server-side span with the mobile user action
4. The result is a single distributed trace spanning device to backend

iOS Implementation

```
// iOS -- tag outgoing web request
let url = URL(string: "https://api.myapp.com/checkout")!
var request = URLRequest(url: url)
if let tag = DTXAction.getRequestTag(for: request) {
    request.addValue(tag, forHTTPHeaderField: "x-dynatrace")
}
let task = URLSession.shared.dataTask(with: request) { data, response, error
in
    // handle response
}
task.resume()
```

Android Implementation

```
// Android -- tag outgoing web request
val url = URL("https://api.myapp.com/checkout")
val connection = url.openConnection() as HttpURLConnection
val tag = Dynatrace.getRequestTag(connection)
if (tag != null) {
    connection.setRequestProperty("x-dynatrace", tag)
}
// proceed with request
```

When to Use Manual Tagging

Scenario	Auto-tagged?	Manual Tagging Needed?
URLSession / HttpURLConnection	Yes (usually)	No
Custom networking libraries (Alamofire, OkHttp)	Depends on version	Often yes
WebSocket connections	No	Yes
gRPC calls	No	Yes
GraphQL over custom transport	No	Yes

> **Note:** Most standard HTTP libraries are auto-instrumented by the SDK. Manual tagging is primarily needed for custom or non-standard network transports.

5. A/B Testing & Feature Flags

Instrumenting A/B tests and feature flags in Dynatrace lets you correlate experiment variants with real user performance and business outcomes. By reporting variant assignments as session properties and business events, you can answer questions like: "Does variant B of the checkout flow have a higher crash rate?" or "Which feature flag configuration drives more conversions?"

Reporting A/B Test Variants

Use session properties to tag the user's session with the assigned variant:

```
// iOS -- report A/B test variant as session property
DTXAction.reportValue(withName: "ab_test_checkout_v2", stringValue:
"variant_b")
```

```
// Android -- report A/B test variant as session property
Dynatrace.reportValue("ab_test_checkout_v2", "variant_b")
```

Reporting Feature Flag States

Send business events when feature flags are evaluated so you can track which users see which features:

```
// iOS -- report feature flag evaluation as business event
let flagAttributes: [String: Any] = [
    "event.type": "com.myapp.feature_flag",
    "feature.name": "dark_mode",
    "feature.variant": "enabled",
    "feature.source": "launchdarkly"
]
Dynatrace.sendBizEvent(type: "com.myapp.feature_flag", attributes:
flagAttributes)
```

Track App Version Adoption Over Time

App version adoption is closely related to feature flag rollouts. Use this query to see how session volume distributes across app versions:

```
// Track app version adoption over time
fetch bizevents, from:-7d
| filter event.provider == "www.dynatrace.com/mobile"
| filter isNotNull(app.version)
| makeTimeseries session_count = countDistinct(dt.rum.session.id), by:
{app.version}, interval:1d
```

6. SDK Performance Optimization

The Dynatrace mobile SDK is designed to be lightweight, but in performance-sensitive applications you may need to fine-tune its behavior. This section covers the key optimization levers.

Optimization Levers

Optimization	Description	Impact
Beacon batching	SDK batches beacons before sending	Reduces network overhead
Action timeout	Tune action close timeout (default 500ms)	Balances accuracy vs. payload size
Excluded URLs	Skip monitoring for analytics/CDN URLs	Reduces beacon volume
Crash-only mode	Set performance collection level to crash-only	Minimum overhead
Sampling	Reduce data collection percentage	Reduces DEM unit consumption

Configuring Excluded URLs

Exclude third-party analytics and CDN URLs that generate noise without providing actionable insight:

```
&lt;!-- Android -- dynatrace.config.xml --&gt;
&lt;monitoring&gt;
    &lt;excludeURLs&gt;
        &lt;exclude pattern="https://analytics.google.com/*" /&gt;
        &lt;exclude pattern="https://cdn.myapp.com/*" /&gt;
        &lt;exclude pattern="https://firebaselogging.googleapis.com/*" /&gt;
    &lt;/excludeURLs&gt;
&lt;/monitoring&gt;
```

Performance Collection Levels

Level	What Is Captured	Overhead	DEM Units
Full	User actions, network requests, crashes, session replay	Highest	Full
User actions	User actions, crashes (no network details)	Medium	Reduced
Crash-only	Crashes and errors only	Minimal	Lowest
Off	Nothing (SDK disabled)	None	None

Sampling Configuration

Sampling reduces the percentage of sessions that are fully monitored. This is useful for high-traffic apps where 100% monitoring is not cost-effective:

- **100% sampling** -- All sessions monitored (default)
- **50% sampling** -- Half of sessions monitored; DEM unit consumption halved
- **10% sampling** -- One in ten sessions monitored; suitable for very high-traffic apps

> **Important:** Sampling is configured in the Dynatrace UI under Application Settings > Data Privacy > Session Replay & Data Collection. It applies to all users of that application configuration.

Query Feature Flag Event Tracking

Use this query to see which feature flags are most actively evaluated and which variants are being served:

```
// Feature flag event tracking
fetch bizevents, from:-24h
| filter event.type == "com.myapp.feature_flag"
| summarize usage_count = count(), by:{feature.name, feature.variant}
| sort usage_count desc
| limit 20
```

7. Multi-App Strategies

Many organizations operate multiple mobile applications -- a customer-facing app, a driver/courier app, an internal admin app, or white-label variants for different brands. Managing Dynatrace monitoring across multiple apps requires careful planning.

Shared vs. Separate Configurations

Strategy	Description	Best For
Separate app configs	Each app has its own Dynatrace application ID and configuration	Apps with distinct user bases and SLOs
Shared app config	Multiple app variants share one configuration	White-label apps with identical codebases
Hybrid	Shared config for variants, separate for distinct apps	Organizations with both scenarios

Multi-App Considerations

- **Naming conventions** -- Use consistent prefixes (e.g., `MyBrand Customer`, `MyBrand Driver`, `MyBrand Admin`) so you can filter and group easily in DQL
- **Shared business events** -- If multiple apps send the same `event.type`, include an `app.name` attribute to distinguish the source
- **Cross-app session linking** -- When a user action in one app triggers backend calls that affect another app, use web request tagging (Section 4) to maintain trace

continuity

- **DEM unit budgeting** -- Each app consumes DEM units independently. Use sampling strategically for high-traffic apps while keeping 100% monitoring for critical apps

Compare Session Volume Across Apps

Use this query to compare daily session volumes across all your mobile applications:

```
// Session volume comparison across all mobile apps
fetch bizevents, from:-7d
| filter event.provider == "www.dynatrace.com/mobile"
| filter isNotNull(dt.rum.session.id)
| makeTimeseries session_count = countDistinct(dt.rum.session.id), by:
{useraction.application}, interval:1d
```

Series Complete

Congratulations on completing all 12 notebooks in the **MOBL (Mobile Monitoring)** series! You now have a comprehensive understanding of Dynatrace mobile RUM -- from foundational concepts to advanced instrumentation techniques.

Full Series Recap

Notebook	Title	Focus
MOBL-01	Mobile Monitoring Fundamentals	Architecture, platforms, entity types, beacon data flow
MOBL-02	SDK Setup -- iOS	CocoaPods/SPM integration, configuration, verification
MOBL-03	SDK Setup -- Android	Gradle integration, configuration, verification
MOBL-04	Cross-Platform Frameworks	Flutter, React Native, Cordova, Xamarin setup
MOBL-05	User Action Tracking	Auto and manual actions, naming rules, session properties
MOBL-06	Crash Reporting	Crash analysis, symbolication, ANR detection
MOBL-07	Network Request Monitoring	HTTP monitoring, error rates, latency analysis
MOBL-08	Session Replay	Visual session replay, privacy masking, debugging
MOBL-09	Performance Analysis	App launch time, Apdex, device/OS segmentation
MOBL-10	Alerting & SLOs	Mobile-specific alerting, SLO configuration, anomaly detection
MOBL-11	Dashboards & Reporting	Mobile dashboards, executive reporting, trend analysis
MOBL-12	Advanced Instrumentation & Optimization	Custom events, request tagging, A/B testing, SDK tuning

What's Next?

- **Apply what you learned** -- Implement custom business events and manual instrumentation in your production apps

- **Explore related series** -- Review the **SPANS** series for distributed tracing or **WFLOW** series for automated alerting workflows
- **Stay current** -- Dynatrace regularly updates the mobile SDK with new capabilities. Check the Dynatrace release notes for the latest features

---